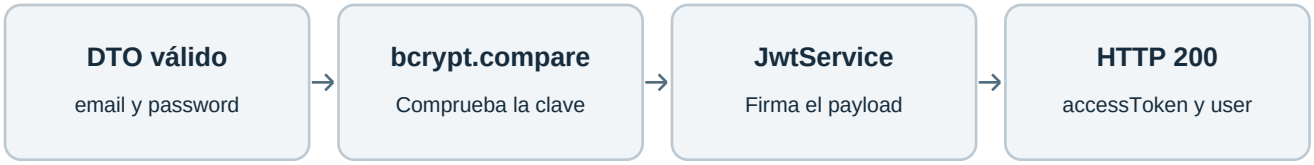
Sesión 17
Emitir tokens JWT en CourseHub

Guía de cambios para aprender la implementación

Robert Alejandro Garcia Reyes · 27 de octubre de 2026

El login ahora entrega una credencial temporal después de comprobar la contraseña. Esta guía compara la base de la sesión 16 con el código final de la sesión 17 y explica por qué se hizo cada cambio.

Resultado: POST /auth/login responde 200 con **accessToken** y **user**. El token dura 15 minutos y solo contiene identidad mínima y tiempos. Una contraseña incorrecta conserva el 401 sin token.



Si bcrypt no coincide: 401 y el flujo termina antes de firmar.

Dónde mirar primero

Archivo	Qué cambió
src/auth/auth.service.ts	Separa validateCredentials, inyecta JwtService y firma al finalizar la validación.
src/auth/auth.module.ts	Registra JwtModule desde ConfigService.
src/auth/types/jwt-payload.type.ts	Define los únicos datos propios que se firman: sub y email.
src/config/environment.ts y .env.example	Exigen un secreto válido y documentan la duración.
test y postman	Comprueban el contrato nuevo y el comportamiento del token.

La sesión 17 emite tokens. La protección de rutas con estrategia y guard corresponde a la sesión 18. Los listados existentes siguen disponibles con 200.

1 Configuración del secreto y la caducidad

Se añadió @nestjs/jwt al proyecto. El secreto se obtiene del entorno; no está escrito dentro de AuthService ni AuthModule. También se preparó un .env local con un valor aleatorio, ignorado por Git.

```
npm install @nestjs/jwt

# .env.example: este marcador NO sirve como secreto efectivo
JWT_SECRET=replace-with-a-long-random-local-secret
JWT_EXPIRES_IN=15m
```

Reglas nuevas en src/config/environment.ts

```
08    JWT_SECRET: Joi.string()
09      .min(32)
10      .pattern(/\S/)
11      .invalid('replace-with-a-long-random-local-secret')
12      .required(),
13    // Require an explicit unit: a bare string such as "900" means milliseconds.
14    JWT_EXPIRES_IN: Joi.string()
15      .pattern(/^[1-9]\d{0,5}[smhd]$/)
16      .default('15m'),
```

JWT_SECRET: es obligatorio, tiene al menos 32 caracteres, no puede ser solo espacios y rechaza el marcador compartido de ejemplo. El valor local se generó con randomBytes(48).toString('hex'): 48 bytes aleatorios representados en 96 caracteres hexadecimales.

JWT_EXPIRES_IN: por defecto usa 15m. Se exigen uno a seis dígitos positivos y una unidad s, m, h o d. Así evitamos que una cadena como 900 se interprete como milisegundos.

Valor	Interpretación
15m	15 minutos; exp - iat = 900 segundos.
1s	Un segundo, útil para la comprobación breve.
900 / 0s / -1m / never	Rechazados antes de iniciar la aplicación.

El esquema informa el nombre de la variable y la regla que falló; no incluye el secreto rechazado en el mensaje. Se comprobó un arranque real sin JWT_SECRET: el proceso terminó con error de configuración antes de abrir la conexión PostgreSQL.

En tu equipo, revisa los datos DATABASE_* del .env para que coincidan con tu servidor PostgreSQL. El entorno de pruebas usó una base temporal y su configuración no sustituye tu conexión habitual.

2 Cómo Nest entrega JwtService al login

AuthModule antes solo importaba UsersModule. Ahora registra el módulo JWT mediante una fábrica asíncrona. Nest obtiene ConfigService, ejecuta la fábrica y crea JwtService con esas opciones.

```
01  import { Module } from '@nestjs/common';
02  import { ConfigModule, ConfigService } from '@nestjs/config';
03  import { JwtModule, type JwtSignOptions } from '@nestjs/jwt';
04  import { UsersModule } from '../users/users.module.js';
05  import { AuthController } from './auth.controller.js';
06  import { AuthService } from './auth.service.js';
07
08  @Module({
09    imports: [
10      UsersModule,
11      JwtModule.registerAsync({
12        imports: [ConfigModule],
13        inject: [ConfigService],
14        useFactory: (config: ConfigService) => ({
15          secret: config.getOrThrow<string>('JWT_SECRET'),
16          signOptions: {
17            algorithm: 'HS256',
18            expiresIn:
19              config.getOrThrow<JwtSignOptions['expiresIn']>('JWT_EXPIRES_IN'),
20          },
21        }),
22      }),
23    ],
24    controllers: [AuthController],
25    providers: [AuthService],
26  })
27  export class AuthModule {}
```

Tipo nuevo de payload

```
01  // Only public identity belongs in the payload. JwtService adds iat and exp.
02  export type JwtPayload = {
03    sub: number;
04    email: string;
05  };
```

Lee estas tres decisiones

registerAsync: permite inyectar ConfigService antes de construir las opciones del módulo. No significa que la contraseña se firme sin validación.

getOrThrow: exige que cada variable esté disponible. El esquema de entorno ya convierte y valida los valores al iniciar la aplicación.

JwtSignOptions['expiresIn']: conserva el tipo que acepta la biblioteca. El esquema Joi es la comprobación real del valor en ejecución. HS256 fija el algoritmo de firma de esta implementación.

3 El cambio principal en AuthService

La lógica existente de bcrypt permanece dentro de validateCredentials. login espera ese resultado y después firma el objeto explícito. Este es el archivo final; los números son las líneas del código.

```
01  import { Injectable, UnauthorizedException } from '@nestjs/common';
02  import { JwtService } from '@nestjs/jwt';
03  import * as bcrypt from 'bcrypt';
04  import { UsersService } from '../users/users.service.js';
05  import type { RegisterDto } from './dto/register.dto.js';
06  import type { LoginDto } from './dto/login.dto.js';
07  import type { JwtPayload } from './types/jwt-payload.type.js';
08
09  @Injectable()
10  export class AuthService {
11    constructor(
12      private readonly usersService: UsersService,
13      private readonly jwtService: JwtService,
14    ) {}
15
16    async register(dto: RegisterDto) {
17      const passwordHash = await bcrypt.hash(dto.password, 12);
18      const user = await this.usersService.create(dto.email, passwordHash);
19      return { id: user.id, email: user.email, createdAt: user.createdAt };
20    }
21
22    async login(dto: LoginDto) {
23      const user = await this.validateCredentials(dto.email, dto.password);
24      const payload: JwtPayload = { sub: user.id, email: user.email };
25
26      return {
27        accessToken: await this.jwtService.signAsync(payload),
28        user,
29      };
30    }
31
32    private async validateCredentials(email: string, password: string) {
33      const user = await this.usersService.findByEmailWithPassword(email);
34      const passwordMatches = user
35        ? await bcrypt.compare(password, user.passwordHash)
36        : false;
37
38      if (!user || !passwordMatches) {
39        throw new UnauthorizedException('Invalid credentials');
40      }
41
42      return { id: user.id, email: user.email, createdAt: user.createdAt };
43    }
44  }
```

Para seguir el flujo: constructor recibe servicios; register conserva su respuesta segura; login construye la carga; validateCredentials busca el hash y lanza 401 si no hay coincidencia. Un throw impide ejecutar signAsync.

4 Antes y después de la respuesta HTTP

Antes, el login de sesión 16 devolvía directamente los datos del usuario. El consumidor leía body.id y body.email. Ahora esos datos se encuentran en body.user y aparece accessToken en la raíz.

Antes de la sesión 17

```
{
  "id": 1,
  "email": "aprendo-jwt@example.com",
  "createdAt": "2026-10-27T18:47:55.228Z"
}
```

Respuesta real de la implementación nueva

```
{
  "accessToken": "[OMITIDO: credencial]",
  "user": {
    "id": 1,
    "email": "aprendo-jwt@example.com",
    "createdAt": "2026-10-27T18:47:55.228Z"
  }
}
```

El texto [OMITIDO: credencial] sustituye el token completo únicamente en el documento y el archivo de evidencia. La API sí devolvió un token real y su firma se verificó durante la ejecución.

Payload leído y verificado localmente

```
{
  "sub": 1,
  "email": "aprendo-jwt@example.com",
  "iat": 1793126876,
  "exp": 1793127776
}
```

sub identifica al usuario y **email** aporta su correo normalizado. **iat** es la emisión y **exp** el vencimiento, en segundos Unix. La diferencia observada fue 900 segundos. createdAt queda en user, fuera del JWT.

Un JWT está firmado y su payload es legible mediante Base64URL. decode solo lo lee; verifyAsync comprueba su firma y vencimiento. No se añaden password, passwordHash, JWT_SECRET ni roles.

5 Qué se comprobó de verdad

La ejecución utilizó HTTP real, bcrypt, JwtService y PostgreSQL 17 en un esquema temporal. Ese esquema se eliminó al terminar. La evidencia pública conserva respuestas y conclusiones, sin contraseñas, hashes, secretos ni tokens completos.

Pruebas automatizadas	Resultado
Unitarias, incluida configuración JWT	31 correctas
HTTP de registro y login	10 correctas
HTTP y criptografía JWT	6 correctas
Integración con PostgreSQL	18 correctas
Arranque completo e2e	1 correcta
Total	66 correctas

Caso observado	Resultado
Registro / correo duplicado	201 / 409
Login correcto	200, accessToken verificable y user seguro
Clave incorrecta / correo inexistente	Mismo 401 público y ninguna firma nueva
DTO inválido	400; no consulta repositorio ni firma
Payload alterado / secreto distinto	La verificación rechaza la firma
JWT_SECRET ausente	El proceso real no inicia
Reinicio de Nest y conexión	La cuenta persiste; nuevo login 200
Cursos / estudiantes / matrículas	Los tres listados responden 200

La prueba de caducidad firma un token con duración 1s y coloca el reloj de verificación en exp. Se obtuvo TokenExpiredError sin esperar tiempo real. El valor normal siguió en 15m. La validación de Bearer mediante HTTP en rutas protegidas será parte de la sesión 18.

Además de las 66 pruebas, el generador de evidencia realizó 14 comprobaciones. Fuentes locales: output/sesion-17-verificacion.json y test/session-17-evidence.mjs.

6 Cómo repetirlo con Postman y la terminal

Importa postman/sesion-17.postman_collection.json. Usa un entorno privado con baseUrl y practicePassword; define una contraseña de práctica de 8 a 72 caracteres y hasta 72 bytes. La colección genera un correo nuevo y ejecuta ocho peticiones en orden.

Paso	Qué observar
1 Registro	201 y datos seguros del usuario.
2 Login válido	200; accessToken y user; payload con cuatro campos.
3 y 4 Credenciales inválidas	401 idéntico para clave incorrecta y correo inexistente.
5 Cuerpo inválido	400 sin token.
6 a 8 Listados anteriores	Cursos, estudiantes y matrículas con 200.

Los scripts decodifican el payload localmente y comprueban sub, email, iat, exp y la duración. No envían el token a páginas externas ni lo guardan en variables de colección. Decodificar aquí no prueba la firma; test:jwt sí la verifica.

Comandos disponibles

```
npm ci
npm run build
npm run lint
npm test
npm run test:auth
npm run test:jwt

# Con PostgreSQL y JWT configurados en el entorno:
npm run test:integration
npm run test:e2e
npm run verify:session17

# Para trabajar con la API:
npm run start:dev
```

verify:session17 crea un esquema de prueba en la base configurada, ejecuta las comprobaciones, guarda JSON sin credenciales y elimina el esquema. El usuario de PostgreSQL necesita permiso para crear esquemas.

La colección de sesión 16 documenta el contrato histórico. Para probar el formato actual usa la de sesión 17. Si acortas la duración, ajusta expectedLifetimeSeconds y restaura 15m al terminar.

7 Repaso para aprender y ubicar la entrega

Preguntas con su explicación

¿Por qué no se firma la entidad completa?

Porque puede contener el hash y campos innecesarios. JwtPayload limita los datos propios a sub y email; el objeto de ejecución también se construye explícitamente.

¿Qué tres datos nunca pondrías en el payload?

La contraseña permite entrar; el hash permite ataques fuera de línea; el secreto JWT permite falsificar tokens. Los tres deben permanecer fuera de la carga.

¿Por qué el token debe expirar?

Para limitar cuánto tiempo puede usarse una credencial. Tener una firma correcta no vuelve vigente un token que ya llegó a exp.

¿Qué pasa cuando cambia el secreto?

Al verificar con el secreto nuevo, la firma de los tokens anteriores deja de coincidir. Durante la prueba se confirmó ese rechazo.

¿Por qué dos logins pueden producir el mismo token?

Si ocurren en el mismo segundo y usan igual identidad, secreto y duración, pueden compartir iat y exp. No se añadió un identificador aleatorio jti en esta sesión.

¿Ya están protegidas las rutas?

Todavía no. Emitir un token no impide acceder a cursos o estudiantes: la sesión 18 añadirá la verificación en las rutas.

Base y archivos de esta entrega

El remoto tenía la sesión 15. La carpeta ya contenía el login de sesión 16 y correcciones anteriores de parámetros en estudiantes y matrículas. Se incluyen como base del código que se probó; no se presentan como requisitos nuevos de JWT.

La guía editable está en docs/sesion-17.md. El PDF está en output/pdf/CoursehubAPI-Sesion-17.pdf y la colección en postman/sesion-17.postman_collection.json. El .env real queda únicamente en el equipo.

Fuente de la actividad

epanchanaf.github.io/nestjs-course/semana-09/sesion-17

Referencia técnica de firma y módulo: github.com/nestjs/jwt. La implementación usa @nestjs/jwt 12.0.3 y sus tipos instalados.